The backend

Read this before changing anything in `supabase/` or `lib/live/`.

Part 1, the basics, is for any developer: what the backend is, how one request travels, where things live, and the four jobs you will actually do. **Part 2, advanced,** is the detail behind it: the security vocabulary, every table, and the rules that have bitten before. Read Part 1 first; go to Part 2 when a task takes you there.

Part 1. The basics

What the backend is

One Supabase (https://supabase.com) project, which is four things you already know by other names:

Piece	What it does here
Postgres	Every table: profiles, pairings, messages, studies, posts, and the rest.
Auth	Sign-in with email and password, and invitations.
Storage	Files: photos, attachments, study files.
Edge functions	Three small server programs, in `supabase/functions/`: `invite` (sends an invitation; it holds the one key that can bypass the rules, so it never runs in a browser), `notify` (sends a notification to a person's devices) and `places` (suggests places as somebody types where to meet).

There is no other server. The app is a website that talks to Supabase directly.

The sample church (the tutorial, "No database connected") uses **no backend at all**: it keeps an invented church in the browser (`lib/demo/store.tsx`). Nothing in this document applies to it.

How one request travels

```
the screen            components/Live*.tsx
   asks for data  ->  lib/live/data.ts        one function per thing, e.g.
listAnnouncements()
   which calls    ->  Supabase, as the signed-in person
   which checks   ->  the database's own rules (row level security)
   and returns    ->  only the rows that person may see
```

The database decides who may see what. Nothing else does. The browser asks plainly; the answer is already correct. If you find yourself filtering rows in JavaScript "for safety", stop: anybody can call Supabase directly with the same public key, so a rule only the browser enforces is not enforced at all. The rule belongs in a migration.

Where things live

```
supabase/migrations/   every table and every rule, in order, one file per change
supabase/functions/    the three edge functions (invite, notify, places)
supabase/seed/         one-off scripts, such as making the first Director
supabase/tests/        SQL checks run against a real database
lib/live/data.ts       every query the app makes, one function per thing
lib/live/session.tsx   who is signed in, and their profile
lib/live/keep-up.ts    which tables each screen listens to for changes
tests/                 one file per rule, each broken on purpose before it is trusted
```

The four jobs you will actually do

1. **Show something new.** Add a function to `lib/live/data.ts` that asks for it, and call it from the screen. If the right people can already read those rows, that is all.
2. **Store something new.** Write a new migration in `supabase/migrations/` : the table, `alter table ... enable row level security` , and a policy for each of select, insert, update and delete that should exist. Then the function in `lib/live/data.ts` , then a test in `tests/` that would fail without the rule.
3. **Change who may see or do something.** Never edit an old migration. Write a new one that drops the old policy and creates the new one. Old files have already run on real churches' databases.
4. **Check your work.** `npm run verify` runs the type checks, the build and every rule test; it must pass. `npm run verify:all` adds the browser walks.

That is enough to work on most features. When a policy will not do what you expect, or you need to know exactly who can read a table, read Part 2.

Part 2. Advanced

Everything in this part was read out of the **live database** on 9 September 2026, not transcribed from the migrations. Where the two ever disagree, the database is the truth and the migration that drifted is the bug.

The one idea

The database decides who may see what. Nothing else does.

Not the browser, not `lib/live/data.ts` , not the screen. Every query the app makes goes through PostgREST as the signed-in person, and row level security returns the rows they are entitled to. A filter written in JavaScript protects nobody: the browser can call PostgREST directly with the same public key, so anything only the client enforces is enforced nowhere.

That is why `lib/live/data.ts` opens with four rules and why its functions look so plain. They ask for what they want. The answer is already correct.

The vocabulary

Almost every policy in the system is written from these predicates. Learn these nine and you can read any policy in the codebase.

Predicate	True when
`is_approved_user()`	you are signed in **and** a Director has approved you
`auth_role()`	your role: `ds` · `dm` · `admin` · `executive`
`my_church_id()`	the church you belong to
`can_access_church(c)`	you belong to `c`, or you oversee it
`manages_church(c)`	you are an admin of `c`, or an executive over it
`leads_church(c)`	leadership of `c`, for the discipline record
`in_pairing(p)`	you are one of the two people in pairing `p`, and still approved
`is_paired_with(x)`	you walk with `x`, either direction
`in_report(r)`	you raised report `r`, may handle it, or were brought into its conversation

They are all `SECURITY DEFINER`, which is deliberate: a policy on `pairings` that read `pairings` directly would re-enter its own policy and Postgres would refuse the query outright with *"infinite recursion detected in policy"*. That happened once, on a sibling deployment, and it took two screens down together. Every cross-table test goes through one of these instead.

Every table, and who may read it

47 tables. **Row level security is on for all 47.** No exceptions, and the verify gate fails if a new one arrives without it.

The people

Table	Who may read	Live
`profiles`	`manages_church` — plus your own row and the people you walk with, through narrower policies	●
`churches`	`can_access_church`	●
`church_executives`	`can_access_church`	
`invites`	`manages_church`	●
`recommendations`	the Guide who made it, or `manages_church`	●
`profile_changes`	`is_paired_with`	●
`office_plans`	**its owner alone**, for every operation: a person's own Sabbath programs and evangelistic meetings, the copy that follows them to their other devices	●

The relationship

Table	Who may read	Live
`pairings`	the two people in it, or `manages_church`	●
`pairing_requests`	the Guide who asked, or `leads_church`	●
`messages`	`in_pairing`	●
`pairing_media`	`in_pairing`	●
`message_reactions`	`my_pairing_ids()` ; written only through `react_to()` ; a suspended account sees none	●
`meetings`	`in_pairing`	●
`journey_events`	the Guide, or `manages_church`	●
`seeker_notes`	**the author alone**	
`follow_ups`	the owner alone	●
`prayer_requests`	whoever wrote it; the one Explorer a Guide's request was written to, while they walk together; a Guide, for their Explorers' own requests	●
`prayer_encouragements`	same as the request it belongs to	●

The library and the studies

Table	Who may read	Live
`materials`	yours, or `can_read_material`	●
`material_shares`	`in_pairing`	●
`material_hides`	yours alone	●
`lesson_series`	published in your church, yours, `manages_church`, or shared to you as a Guide	●
`lessons`	through the series they belong to	●
`lesson_files`	through the lesson	●
`lesson_reads`	`may_see_reading`	●
`lesson_assignments`	`in_pairing`	●
`lesson_guide_shares`	`manages_church`, or the Guide it was shared with	

The church's voice

Table	Who may read	Live
`announcements`	your church	●
`blog_posts`	yours, or `can_read_post`	●
`blog_audience`	the author of the post	
`guide_room_messages`	`in_guide_room`	●
`guilds` · `guild_members`	your church, or leadership over it	●
`notifications`	yours alone	●

Safeguarding

Table	Who may read	Live
`reports` · `report_files`	whoever **may handle** that report (`may_handle_report`: a leader of its church, never the person it is about, and only an Executive Director when it is about a leader); a report also by whoever raised it. Nobody writes to `reports` directly	●
`report_messages`	`in_report`: who raised it, who may handle it, and the person it is about once brought in	●
`discipline_log`	`leads_church`	●
`feedback`	`manages_church`, or its author	●
`activity_record`	`manages_church`	

The nine tables with no policy at all

· `library_activity` · `library_blocks` · `message_revisions` ·

RLS on, zero policies, which in Postgres means **deny everything**. Nothing reads these directly. They are reached through `SECURITY DEFINER` functions that apply their own rule and often return *less* than the row holds — the guild feed computes an `author_label` rather than handing over `author_id`, so it decides how much of a writer's identity each reader sees.

holds what a message said before it was edited or taken back, and it is unreadable by everybody — including the two people in that conversation. Leadership reads it through a definer function when a report is being looked at, and nowhere else.

This is a deliberate pattern, and it has one sharp consequence.

> A table with no policy can never be watched over realtime. Realtime evaluates the same policies as a `SELECT`, so it delivers to nobody — and the screen looks perfectly wired while staying frozen.

That trap has been walked into three times. `tests/every-room-keeps-up.mjs` now refuses a `KEEP_UP_` set that names a table with no read policy, which is why the library's record watches `materials` and `material_shares` — the two tables whose triggers *write* its rows — instead of the record itself.

The Guild Room's wall was the last screen that still reloaded, for the same reason: the only repair that would obviously work — a read policy on

thing the label exists to do. It is live as of `20260909180000`, and the way it got there is the fourth option in rule 3 below.

`guild_wall_pulse` **is a cause table that was written on purpose.** One row per guild, holding only that its wall changed and how many times: no author, no body, no post id. Triggers on the posts and on the amens bump it, its read policy calls `private.active_guild_member` — the feed's own membership test, called rather than restated — and the screen re-asks `list_guild_activity`, which redacts exactly as before. The raw row never leaves the database, and both guild tables keep RLS on with no read policy and no browser grant.

When there is no cause table to watch, that is not always the end of it. One can be built, provided it carries strictly less than the thing it reports on.

Where the rules live

```
supabase/migrations/     the only place schema or policy changes exist
supabase/functions/      the edge functions, which hold the service key
lib/live/data.ts         every browser query, one function per thing
lib/live/keep-up.ts      which tables each screen listens to
tests/                   one file per rule, each broken on purpose before trust
scripts/verify.mjs       the gate: typecheck, build, and every test above
```

Six rules that bite

1. **Migrations are append-only.** They have run against a live database with real people in it. Fixing a migration means writing the next one; editing a file that has already run puts the repository and the database into a disagreement nobody can see.

1. **A definer function must authorise its own caller.** It runs as its owner, so RLS does not protect it. Every one that *acts* — `suspend_member`, `bring_subject_into_report`, `remove_member_by_leader` — checks the caller first. That check is the only thing standing there.

1. **A policy decides WHICH ROWS. A grant decides WHICH COLUMNS.** RLS cannot express "only this column", and every attempt to make it try has been a hole. `messages_mark` existed so the recipient could stamp `read_at`; because it said nothing about columns, it also let either person in a pairing silently rewrite the other's words. An audit for the same shape found three more: a Guide could rewrite their Explorer's prayer request and publish a private one to the whole church, and either party could rewrite the time, place and joining link of a meeting the other had already confirmed.

The fix is a column privilege — `revoke update`, then `grant update (that_one_column)` — and the two compose cleanly. Before adding an UPDATE policy, ask what the app actually writes to that table, and grant exactly that. `tests/the-browser-writes-only-what-the-app-writes.mjs` holds the list and fails if a later migration hands a whole row back.

1. **Role is not church.** Two families of helper, and they are not interchangeable:

Helper	Checks
`is_admin()` · `is_executive()`	the caller's role. **No church.**
`leads_church(c)` · `manages_church(c)`	the role **and** that church

Anything that takes a member id and acts on them wants the second. Both guardian-consent functions used the first, so a Director of one church could record or withdraw guardian consent for a minor in another — the two most safeguarding-sensitive functions in the app. Use `is_admin()` only where the action has no target, such as creating a church.

1. **A write that wakes every screen needs a ceiling.** Realtime turns one insert into one piece of work per open app: cheap for whoever writes, multiplied for the server. `private.hold_the_pace` is a BEFORE INSERT trigger on the three tables a person can write into a room — the conversation, the Guild wall, the Guides' room — refusing past **40 a minute per account**, which is two a second sustained and far above anything a person

types. It is set to catch a script, not a member: a limit that occasionally catches a real person is a limit that gets removed after the first complaint. It bounds the machine and not the behaviour — forty unkind messages a minute is still a matter for the report route and a Director.

The other half is on the client. Anything watching a table goes through `useKeepUp`, which settles a burst into one reload. A raw channel with no debounce is an amplifier, and the chat dock shipped as one for a day.

1. **Never widen a policy to make a screen convenient.** The screen is the cheaper thing to change. Every time this has come up the answer has been to watch a different table, show a different card, accept a reload — or write a cause table that carries no identity and watch that instead. The Guild wall is the worked example: it is live, and `guild_activity_posts` is no more readable than it was the day the room shipped.

How to satisfy yourself it is true

```
npm run verify                      # typecheck, build, and every guardrail
node tests/every-room-keeps-up.mjs  # the realtime rules, on their own
```

And against a live database, the two questions worth asking after any change:

```
-- Nothing may be readable by accident.
select relname from pg_class c join pg_namespace n on n.oid = c.relnamespace
where n.nspname = 'public' and c.relkind = 'r' and not c.relrowsecurity;

-- Published but unreadable: in the publication, no read policy. These deliver
-- to nobody. That is fine when nothing watches them and a bug when something does.
select p.tablename from pg_publication_tables p
join pg_class c on c.relname = p.tablename
left join pg_policy pol on pol.polrelid = c.oid and pol.polcmd in ('r','*')
where p.pubname = 'supabase_realtime' and p.schemaname = 'public'
group by p.tablename having count(pol.polname) = 0;
```

The first returns nothing, and has since the beginning. The second returns five rows, and that is expected — this page used to say both came back empty, which anybody who ran the second query found out was untrue in about a second:

```
blog_views · guild_activity_amens · guild_activity_posts
library_activity · library_blocks
```

They sit in the publication and deliver to nobody, because RLS filters realtime exactly as it filters a `SELECT`. That is harmless while no screen watches them — nothing in `lib/live/keep-up.ts` names one — and it is why the second query is not the question worth asking. This is:

```
-- A table a screen WATCHES that cannot be read is the actual fault: the screen
-- looks wired and stays frozen. Cross-check the list against KEEP_UP_ sets.
select p.tablename from pg_publication_tables p
join pg_class c on c.relname = p.tablename
left join pg_policy pol on pol.polrelid = c.oid and pol.polcmd in ('r','*')
where p.pubname = 'supabase_realtime' and p.schemaname = 'public'
group by p.tablename having count(pol.polname) = 0;
-- ...then: grep -o "'[a-z_]*'" lib/live/keep-up.ts | sort -u
```

reason to trust it rather than this paragraph. The second returned five rows on 9 September 2026, and that is the bug this page was written after fixing.